Mancala Game

Kasra Sabertehrani
kasra.sabertehrani@studio.unibo.it

Igor Chukarin
igor.chukarin@studio.unibo.it

April 2026

Abstract

This report presents the development of a Mancala game application using Domain-Driven Design (DDD) principles to achieve a clean and maintainable architecture. Instead of focusing on technology first, the system is designed around the game domain, capturing its rules and behavior directly in the domain model.

In addition, the report describes the DevOps practices adopted in the project, including Continuous Integration and Continuous Deployment (CI/CD) pipelines implemented with Docker and GitHub Actions to ensure code quality, automated testing, and reliable delivery.

The resulting system demonstrates how combining DDD and modern DevOps practices leads to a robust, scalable, and well-structured application.

Contents

1 Domain driven design **3**
1.1 The Ubiquitous Language . . . 3
1.2 Bounded Contexts . . . 4
1.3 Tactical Design . . . 4
1.4 Domain Events . . . 6
1.5 Custom Exceptions . . . 6
1.6 DDD class mapping . . . 7

2 Multi-Platform Interoperability **9**
2.1 Defining the Target Platforms . . . 9
2.2 The Interoperability Challenge: Technological Silos . . . 9
2.3 Architectural Strategy Comparison . . . 9
2.4 Bridging the Silos: Protocol Implementation . . . 10

3 DevOps Practices **11**
3.1 Workflow Organization . . . 11
3.2 Build Automation . . . 11
3.3 Self-Testing Code . . . 12
3.4 Code Quality Control . . . 12
3.5 Continuous Integration . . . 13
3.6 Continuous Delivery . . . 13
3.7 Continuous Deployment . . . 14
3.8 Continuous Monitoring . . . 14

4 Conclusion **15**

1 Domain driven design

1.1 The Ubiquitous Language

The ubiquitous Language is the shared vocabulary between developers, game experts, and stakeholders. Below we provided two tables that holds the vocabulary used in Mancala, their definition, and our representation of it in the code.

1.1.1 Domain Vocabulary

Domain vocabulary designed for the project is present in Table 1.

Domain Term	Definition	Java Representation
Pit	A small cup on the board; holds stones	`Pit` class with `int stones` field
Store	The large pit at the end of each side; used for scoring	Special `Pit` at indices 6 & 13 on the board
Sow	The action of picking stones from a pit and distributing them counter-clockwise	`Board.sowStones(int pitIndex)`
Capture	If your last distributed stone lands in your empty pit and the opposite pit has stones, you capture all those stones	`Board.attemptCapture(boolean isPlayer1, int lastIndex, int myStoreIndex)` returns boolean
Free Turn	If your last distributed stone lands in your store, you get to play again	`lastMoveGrantedFreeTurn` boolean property
Side Empty	When a player's side has no stones left, the game ends	`Board.isSideEmpty(int from, int to)`
Game Over	The state when at least one player's side is empty and the game has concluded	`GameStatus.GAME_OVER`
Room	A container for one game between exactly two players	`Room` aggregate root
Absent Player	A player whose network connection dropped during an active game	`absentPlayerId` reference within the `Game` entity
Grace Period	30-second window where a disconnected player can rejoin	`MATCH_SUSPENDED` status
Returned Player	A disconnected player who successfully rejoins the room within the allotted grace period	`PlayerReturnedEvent` domain event
Forfeit	A player abandons the game or fails to reconnect in time; opponent wins	`PlayerForfeitedEvent` domain event

Table 1: Mancala domain vocabulary

This vocabulary was used for classes and methods naming during development.

1.1.2 Game State Transitions

The game progresses through distinct states is represented in Listing 1.

Listing 1: GameStatus Enum - The Game's Lifecycle

```
public enum GameStatus {
    WAITING_FOR_PLAYER_2,   // Room created, opponent hasn't joined yet
    PLAYER_1_TURN,          // Player 1 is currently picking a pit
    PLAYER_2_TURN,          // Player 2 is currently picking a pit
    MATCH_SUSPENDED,        // A player disconnected (grace period
        active)
    GAME_OVER,              // One side is empty; game concluded
    FORFEIT                 // A player abandoned; game concluded
}
```

Using domain language in game states allows for tracking the current state of the system for people not related to the development, but who understand the context.

1.2 Bounded Contexts

The Mancala project operates within one primary bounded context: the «Game Context», which represents the core of the game. This context is responsible for managing game rules, enforcing player turns, maintaining the board state, validating moves, and generating domain events.

All code in the «com.mancalagame.domain» package belongs to this context. Code outside this package (controllers, repositories, event broadcasters) acts as a bridge between the external world and the domain.

1.3 Tactical Design

Tactical patterns are concrete DDD implementations. This section shows how Mancala's Java classes map to tactical patterns.

1.3.1 Aggregate - Room

An aggregate is a cluster of domain objects (entities and value objects) treated as a single, consistent unit. The aggregate has a root—a single entry point for all modifications. Room is the aggregate root because:

- It has a global identity: RoomId
- All game modifications happen through it
- It maintains consistency: validates players, records events
- It enforces business rules: ensures valid player count, turn validity

Critical constraint: External code cannot directly modify Game or Board. They are internal to the aggregate. All changes must go through Room.

1.3.2 Entity - Game

An entity has an identity and a lifecycle. Game is an entity—it exists within the Room aggregate and manages game rules. Game's responsibilities are:

- Rule enforcement: Validates moves, checks game state
- Turn management: Switches between players, grants free turns
- End-game detection: Recognizes when one side is empty
- State tracking: Maintains game status and absent player info

1.3.3 Entity - Board

Board is an entity that encapsulates the board's physical mechanics - the rules about distributing stones. Board's responsibilities are:

- Sowing stones counter-clockwise
- Detecting capture opportunities
- Enforcing stone distribution rules
- Providing game state queries (side empty, stones at index)

1.3.4 Entity - Player

Player is an entity representing a person in the game. Player's role: Mainly a data container for identity and name.

1.3.5 Value Object - Pit

A value object is immutable and identified by its value, not by identity. Pit is a simple value object because:

- No global identity needed - two pits with 4 stones are equivalent
- Encapsulates stone count logic
- Immutable semantics (replaceable without side effects)

1.3.6 Value Objects - PlayerId & RoomId

Identity Value Objects wrap primitive values with type safety and validation. Advantages of this approach:

- Type safety: Cannot swap RoomId for PlayerId
- Validation: Guarantees non-null, non-empty IDs
- Intent: Code readers immediately understand what kind of ID is used
- Serialization: @JsonValue enables clean JSON representation

1.4 Domain Events

Domain Events are immutable records of what happened in the domain. They are the facts of the game. Every domain event:

- Represents something that already happened
- Is immutable (cannot be changed after creation)
- Belongs to exactly one room (aggregate)
- Has a timestamp (audit trail)

1.4.1 MoveMadeEvent

When a player makes a valid move, this event records the facts:

- The fact: A move was made (not the detailed board state)
- Special rules triggered: Did a capture or free turn happen?
- State transition: What status does the game have now?

1.4.2 PlayerLeftTableEvent

When a player's network disconnects this event signals: "This player is now absent. Grace period begins."

1.4.3 PlayerReturnedEvent

When a disconnected player rejoins within the grace period this event signals: "Player is back. Game resumes from suspended state."

1.4.4 PlayerJoinedEvent

When Player 2 joins the room this event signals: "Game now has two players. Game state transitions to PLAYER_1_TURN."

1.4.5 PlayerForfeitedEvent

When a player abandons the game this event signals: "Game is over. The other player wins by forfeit."

1.5 Custom Exceptions

Custom exceptions are used to protect the domain. They are part of the ubiquitous language. They express domain invariants and constraints.

All domain exceptions extend DomainException class, making them easily distinguishable from technical exceptions.

The domain employs custom exceptions to enforce game rules, ensure only registered participants can play, and safely handle requests for non-existent games. These exceptions utilize domain-specific language and fail fast to protect the game state:

- InvalidGameStateException: Enforces the rules of Mancala. It is thrown by the Game class when illegal moves are attempted (e.g., playing out of turn, selecting an empty or invalid pit, or moving after the game has ended). Its error messages use plain, game-specific language rather than technical jargon.
- InvalidPlayerException: Enforces authorization and room registration. It is thrown when an unrecognized, null, or unauthorized player attempts to interact with a specific game room. It captures the offending player's ID to help with debugging and error handling.
- RoomNotFoundException: Ensures the system fails fast when a requested context is missing. It is thrown when a player tries to access, join, or modify a room that does not exist or has already been closed, capturing the invalid room ID for diagnostics instead of allowing null behavior.

Each exception name speaks the language of the domain. General exceptions are not used.

1.6 DDD class mapping

All the classes discussed above, their pattern and responsibilities are represented in the Table 2.

Java Class	**DDD Pattern**	**Domain Concept**	**Responsibility**
`Room`	Aggregate Root	Game Room	Maintain consistency, record events
`Game`	Entity	Game	Enforce rules, manage turns, detect end
`Board`	Entity	Board	Stone mechanics, captures, side queries
`Player`	Entity	Player	Represent participant
`Pit`	Value Object	Pit	Encapsulate stone count
`PlayerId`	Value Object	Player ID	Type-safe player identification
`RoomId`	Value Object	Room ID	Type-safe room identification
`MoveMadeEvent`	Domain Event	Move Made	Record move with special rules
`PlayerLeftTableEvent`	Domain Event	Disconnect	Record player absence
`PlayerReturnedEvent`	Domain Event	Reconnect	Record player return
`PlayerJoinedEvent`	Domain Event	Game Start	Record player joining
`PlayerForfeitedEvent`	Domain Event	Forfeit	Record game abandonment
`InvalidGameStateException`	Custom Exception	Rule Violation	Signal invalid move
`InvalidPlayerException`	Custom Exception	Unknown Player	Signal unauthorized player
`RoomNotFoundException`	Custom Exception	Missing Room	Signal non-existent game

Table 2: DDD Class Mapping

Figure 1 illustrates the core Domain-Driven Design architecture, establishing the Room as the Aggregate Root that securely encapsulates the internal game entities and value objects. It demonstrates how the aggregate maintains strict boundary consistency by throwing domain-specific exceptions to enforce rules and emitting domain events to broadcast state changes.

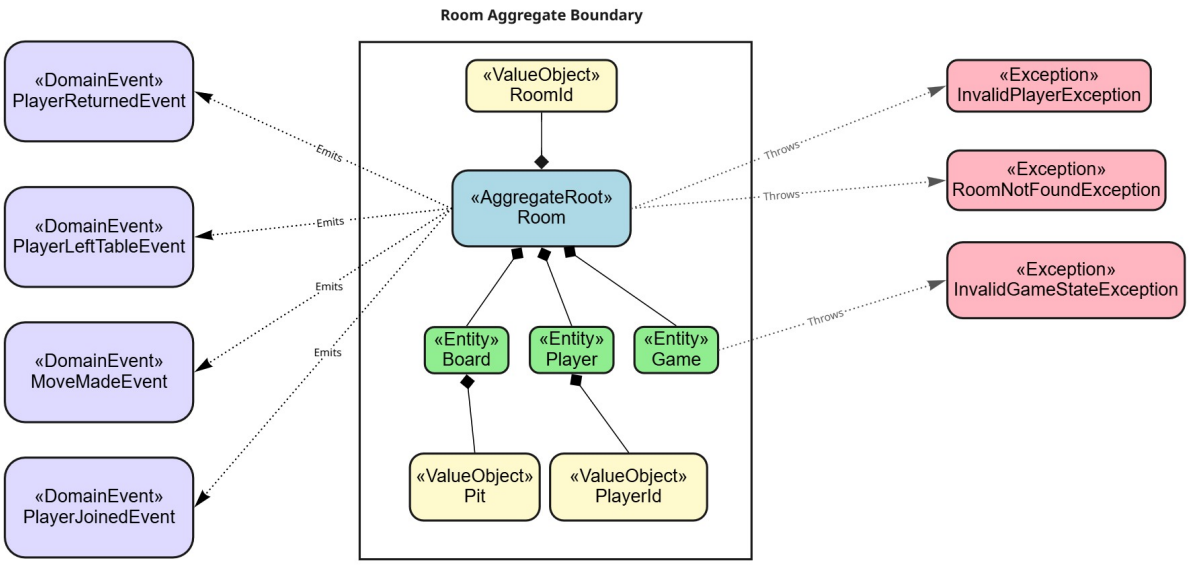

Figure 1: Domain Model Diagram - Class Relationships and Patterns

Domain-Driven Design ensures a reliable and well-structured core of the system, while fostering maintainability and long-term reliability.

2 Multi-Platform Interoperability

A software platform is defined as anything having an API that enables the writing of applications, combined with a runtime that supports the execution of those applications. This section outlines how Mancala Online resolves the challenges of multi-platform programming through a distributed, protocol-driven architecture.

2.1 Defining the Target Platforms

Mancala Online is deployed across two completely distinct platforms, each chosen for its specific strengths in the client-server architecture:

- Platform 1 (The JVM): The Java Virtual Machine serves as the authoritative backend engine. It provides the robust runtime, threading models, and standard libraries (via Spring Boot) necessary to enforce the complex Domain-Driven Design rules, manage game states, and process concurrent multiplayer actions.
- Platform 2 (The Browser / V8 Engine): The web browser (specifically its JavaScript runtime) serves as the frontend platform. It provides the API (DOM) and execution environment required to render the interactive graphical user interface, handle client-side input, and execute animations.

2.2 The Interoperability Challenge: Technological Silos

A significant architectural challenge arises because the JVM and the JavaScript V8 engine act as strict technological silos. Software components within these silos have inherently poor external interoperability. Java bytecode cannot be natively executed within a web browser, and JavaScript cannot natively interact with Spring Boot server threads.

Connecting these two platforms requires crossing a massive abstraction gap—the space between the client's need for real-time UI updates and the server's need for strict, centralized rule enforcement.

2.3 Architectural Strategy Comparison

Traditional multi-platform programming often relies on code-sharing strategies to bridge silos. Two common approaches are:

1. Write once, build anywhere: Using a super-language (like Kotlin Multiplatform) to write common code that is compiled separately for both the JVM and JavaScript environments.
2. Write first, wrap elsewhere: Writing the implementation in a primary platform (like Java) and using wrapping technologies (like JPype) to execute that code in a secondary runtime (like Python).

Our Strategy: Network-Driven Interoperability. Rather than sharing source code across platforms, Mancala Online achieves interoperability by sharing state over standard network protocols. Sharing domain logic code to the frontend would violate our Hexagonal Architecture boundaries and expose authoritative game rules to client-side manipulation.

By keeping the domain strictly isolated on the JVM, we ensure security and consistency, utilizing distributed systems architecture as our multi-platform bridge.

2.4 Bridging the Silos: Protocol Implementation

To allow the JVM and JavaScript platforms to seamlessly interoperate, we employ a dual-protocol networking strategy, using JSON as the universal data-interchange format.

Protocol	**Endpoint Pattern**	**Platform Interoperability Role**
HTTP REST	`/api/rooms`	Handles stateless, one-off commands such as room creation and matchmaking. The JS platform sends commands; the JVM platform executes them and returns the initial state.
STOMP / WS	`/topic/room/{id}`	Maintains a persistent, stateful connection. The JVM broadcasts real-time `Room` aggregate updates; the JS platform listens and reacts instantly.
STOMP / WS	`/topic/.../events`	The JVM broadcasts specific Domain Events (e.g., `MoveMadeEvent`); the JS platform triggers platform-specific UI animations based on these facts.

Table 3: Network Protocols Bridging the JVM and JS Platforms

Full REST API documentation is available via Swagger UI and can be accessed at: http://localhost:8080/swagger-ui/index.html after the application is started.

Full websocket API documentation is available at: http://localhost:8080/websocket.html after the application is started.

3 DevOps Practices

This section outlines the DevOps practices used in the project, including workflow organization, CI/CD automation, and approaches to ensuring code quality and reliable deployment.

3.1 Workflow Organization

The master branch is protected to prevent unreviewed code from reaching production. All changes must be merged via pull requests.

When a pull request is created, a CI job of the pipeline is executed. It verifies that the code passes unit tests, performs code quality analysis, and ensures that the build completes successfully. After this, the pull request must be reviewed by at least one team member before it can be merged.

After changes are pushed to the master branch, the «publish» and «deploy» jobs of the pipeline are executed. The «publish» job builds Docker image and pushes it to the GitHub Container Registry. The «deploy» job pulls the image to the server and starts a container based on it. Overall workflow is illustrated in Figure 2.

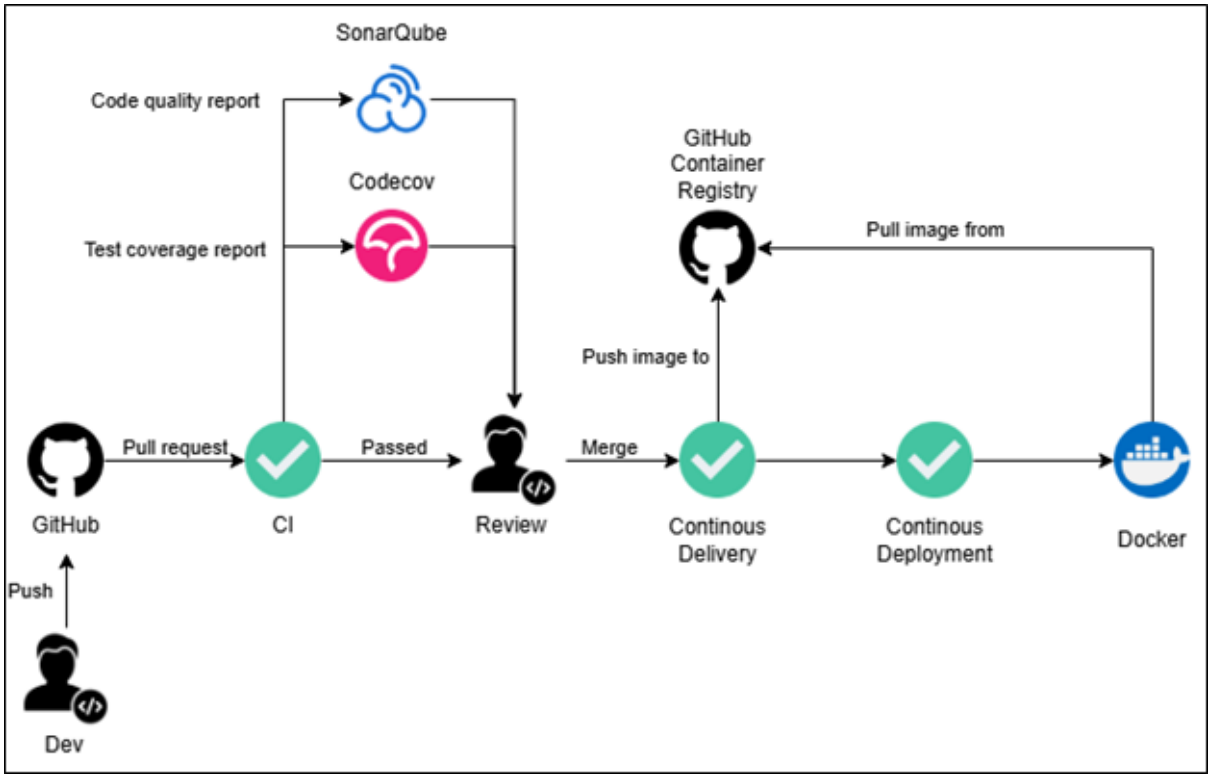

Figure 2: Workflow

Further details are discussed in the following sections.

3.2 Build Automation

Build automation is implemented using Maven and Docker. It ensures that the application is compiled and packaged. The build process includes resolving dependencies,

compiling the source code, and packaging it using Maven. The result is a deployable JAR artifact.

In addition to the application build, a Docker image is created using a Dockerfile. A multi-stage Dockerfile is used to separate the build and runtime environments, reducing the size of the final image.

To improve reproducibility, fixed versions of key dependencies, plugins, and the Java runtime are used in pom.xml file.

As a result, we have Docker image that encapsulates the application and its runtime environment.

3.3 Self-Testing Code

Self-testing code is implemented through unit tests. The project uses JUnit for testing and Mockito for mocking dependencies.

Code coverage is measured using JaCoCo, providing information about how much of the codebase is tested. Codecov is used to generate reports in the CI job of the pipeline.

During pull requests, we can see whether new changes are covered by tests and to what extent.

3.4 Code Quality Control

Code quality control in the project is ensured through a combination of code review practices and automated analysis tools. All changes must be submitted via pull requests, and each pull request must be reviewed and approved by at least one team member before being merged.

In addition to manual review, automated code quality analysis is performed using SonarQube. This tool analyzes the codebase to detect potential bugs, code smells, security and maintainability issues. The progress of code quality can be see in Figure 3.

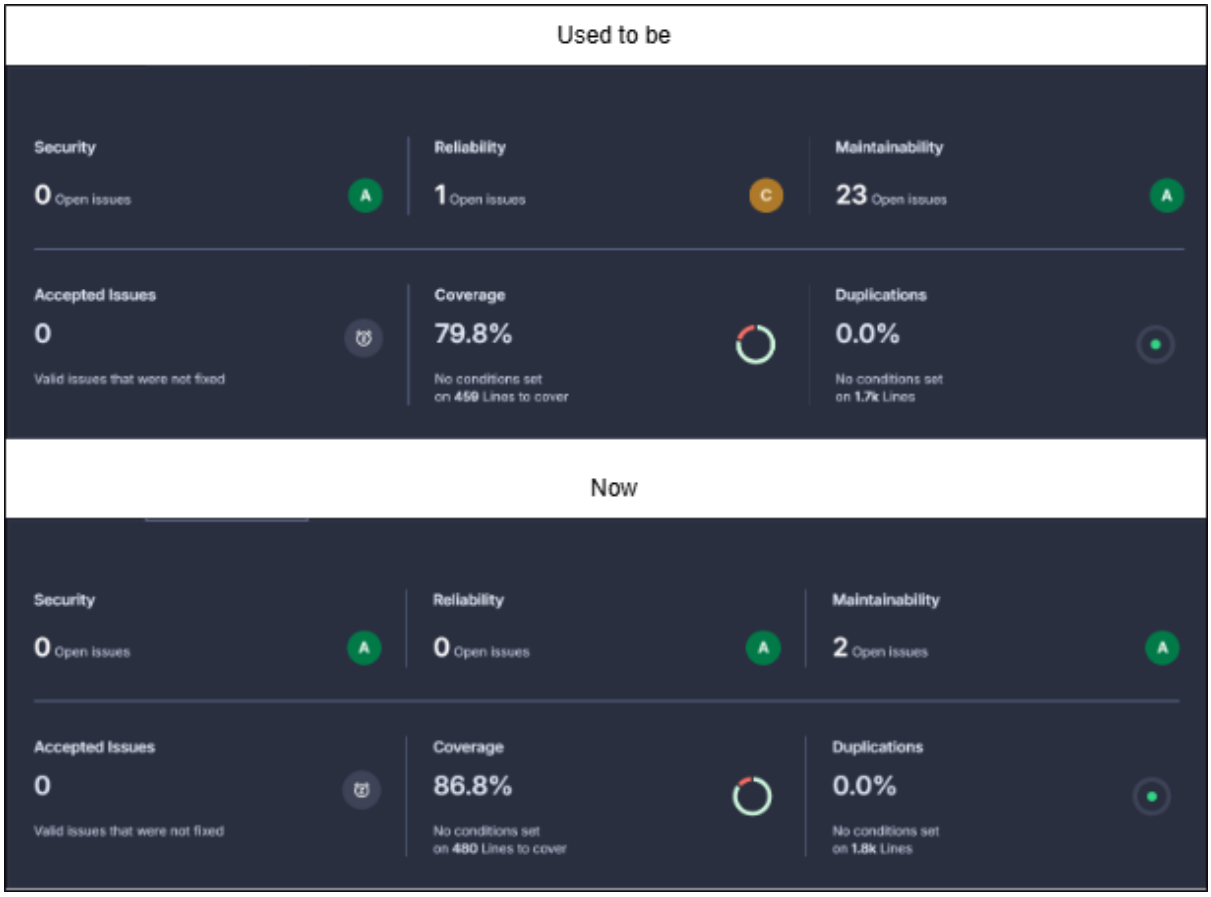

Figure 3: Code quality progress

SonarQube highlighted an important issue that we had with synchronization, allowing us to fix it.

3.5 Continuous Integration

The CI process is automatically triggered on pull requests and pushes to the master branch, ensuring that all changes are validated before integration into the main codebase.

The CI job consists of several steps. First, the source code is checked out from the repository. Then, the build environment is prepared by setting up Java 17 and enabling Maven dependency caching. After that, the Maven Wrapper is made executable, and the project is built and tested using the Maven clean verify lifecycle.

Static code quality analysis is performed using SonarQube. Test coverage is measured using JaCoCo and reported via Codecov.

Finally, the pipeline verifies that the Docker image can be successfully built, ensuring that the application is ready for containerization.

This process ensures early detection of errors, maintains code quality, and guarantees that only validated changes are integrated into the main branch.

3.6 Continuous Delivery

Continuous Delivery is implemented through the «publish» job of the pipeline. This stage includes authentication with the container registry, building a Docker image, tagging

it with both the commit SHA and the latest tag, and pushing it to the GitHub Container Registry.

As a result, a versioned and deployable artifact is produced and made available for deployment.

3.7 Continuous Deployment

Continuous Deployment is implemented through the «deploy» job of the pipeline. After successful completion of the build and image publication stages, the application is automatically deployed to the production environment.

The deployment process is executed over SSH using a remote server. The latest Docker image is pulled from the GitHub Container Registry. The currently running container is stopped and removed, and a new container is started based on the updated image.

This approach enables automated deployment of the application after each successful change.

3.8 Continuous Monitoring

Continuous Monitoring is implemented through infrastructure monitoring and application logging. Server-level metrics are monitored using tools provided by the hosting provider, Hetzner, allowing tracking of resource usage such as CPU, memory, and network activity.

In addition, application logs are used to monitor system behavior and detect potential issues. Logs provide insights into runtime events, errors, and application performance.

This approach enables basic observability of the system and helps identify problems in the production environment.

4 Conclusion

This project demonstrates the benefits of applying modern software engineering practices in the development process.

Domain-Driven Design improves maintainability by ensuring that classes have clear responsibilities and low coupling. As a result, the system becomes easier to understand, test, and extend with new functionality.

The use of two target platforms highlights the effectiveness of a distributed, protocol-based architecture. By separating the JVM-based backend from the browser-based frontend, the system achieves both flexibility and strong control over domain logic.

DevOps practices further enhance the development process by enabling fast and reliable feedback through automated pipelines. Continuous Integration ensures that code is regularly tested and analyzed, while Continuous Deployment allows for rapid and consistent delivery to production. As a result, issues are detected and resolved early, and only high-quality changes are integrated into the system.

Overall, the combination of DDD, multi-platform architecture, and DevOps practices results in a system that is maintainable, scalable, and reliable.